For Project 4, we were tasked with comparing the custom kinodynamic planner RG-RRT (by Shkolnik et al.), with RRT and KPIECE1 in both a pendulum system and a car system. We used OMPL and the RG-RRT publication to create both environments separately, define RG-RRT and implement all three planners, and compare their results via benchmarking and graphical analysis of trajectory data. In both systems, we used a bounded RealVectorControlSpace. By default, the reachability set is found by propagating 11 evenly-spaced control values: torque for the pendulum and heading velocity for the car.

Torque-Controlled Pendulum System:

In this system, the pendulum is a kinodynamic problem that consists of a rigid link that rotates around a fixed pivot, translation to a 2-dimensional state space of angular position and angular velocity. The system was tested with the planners RRT, KPIECE1, and RG-RRT, and generated the following trajectory results:

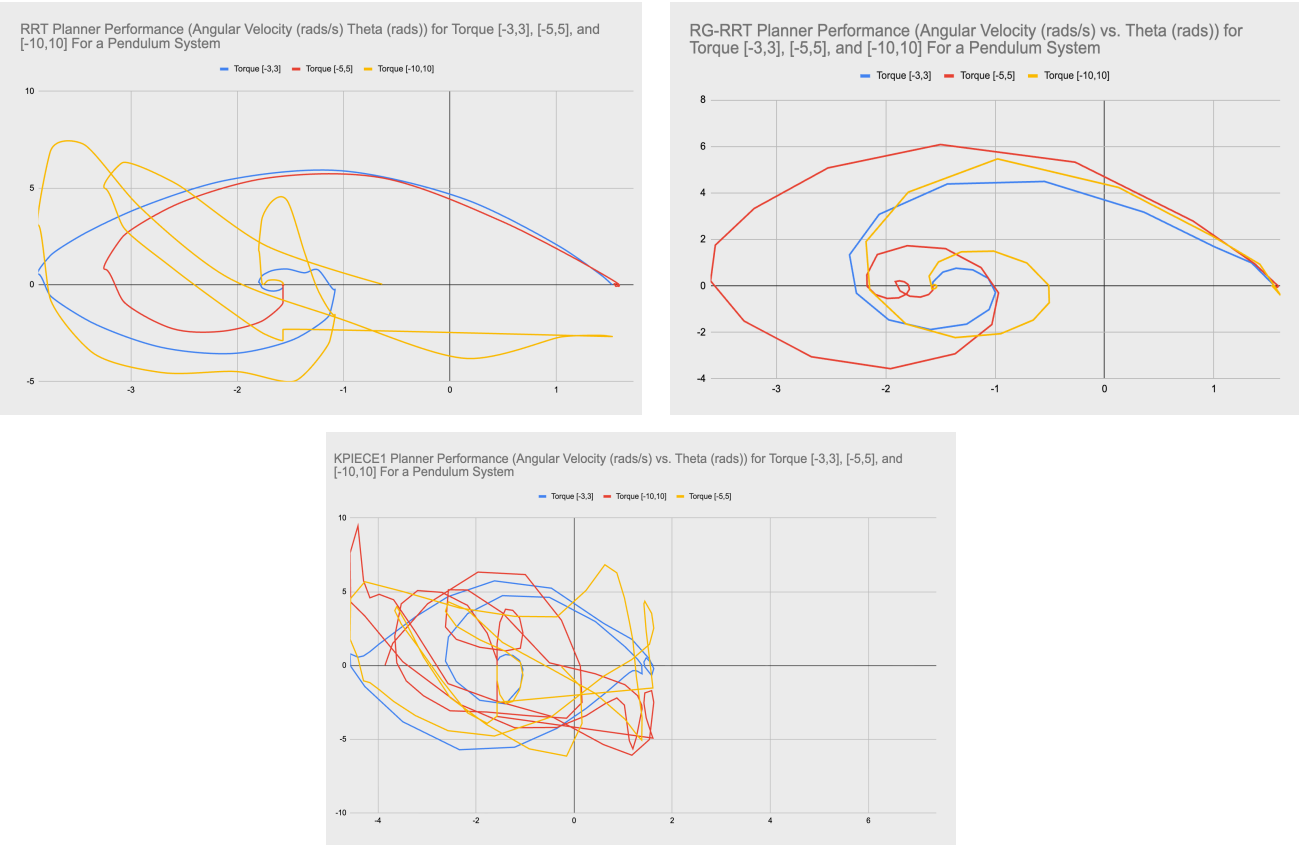


Figure 1: Graphical Presentation of RRT (top left), KPIECE1 (top right), and RG-RRT (bottom) Planner Performance (Theta (rads) vs. Angular Velocity (rads/s)) for Torque [-3,3], [-5,5], and [-10,10] For a Pendulum System.

In the RRT graphs, the lower torques of [-3,3] and [-5,5] present graphs that are relatively smooth and contain minimal loops. However, the higher torque of [-10,10] generated an erratic trajectory with spikes. In the KPIECE1 graphs, all three torques generated relatively rough results, but again the lower torque of [-3,3] presented the cleanest trajectory, whereas the [-10,10] curve was the spikiest of the torques. Here, the trajectories showed significantly sharper transitions. Additionally, the [-10,10] torque trajectory shows that the system reached an angular velocity near the program's set 10 rads/s maximum.

The main difference between the behaviors of the two pre-defined planners in the different torques is that with higher torques, RRT visibly changes drastically because the high constraints in the control space increase the size of the tree's search space. On the other hand, KPIECE1 maintains its rough behavior but changes quantitatively, as the cells are always covered but higher torques cause farther bounds of angular velocity.

In contrast, RG-RRT produced the cleanest trajectories overall. These results can be explained by RG-RRT's reachability-guided rejection filtering, which allows the planner to maintain a clear reachable set. Thus, RG-RRT does not attempt to drive the system into unreachable states and avoids computational overload and erratic trajectories. In the [-3,3] torque, the trajectory is operating with underactuation, and the planner therefore executes precise loops to build momentum. At [-5,5], the planner uses an extreme backwards swing (-3.6 rads/sec), to reach a high peak angular velocity. Lastly, at [-10,10], the planner is working with higher power, and thus avoids extreme swings and takes a more direct path.
To directly compare the performance of KPIECE1, RRT, and RG-RRT, we visualized 20 runs of benchmarking data using Planner Arena. Below are the comparisons over the different cited metrics.

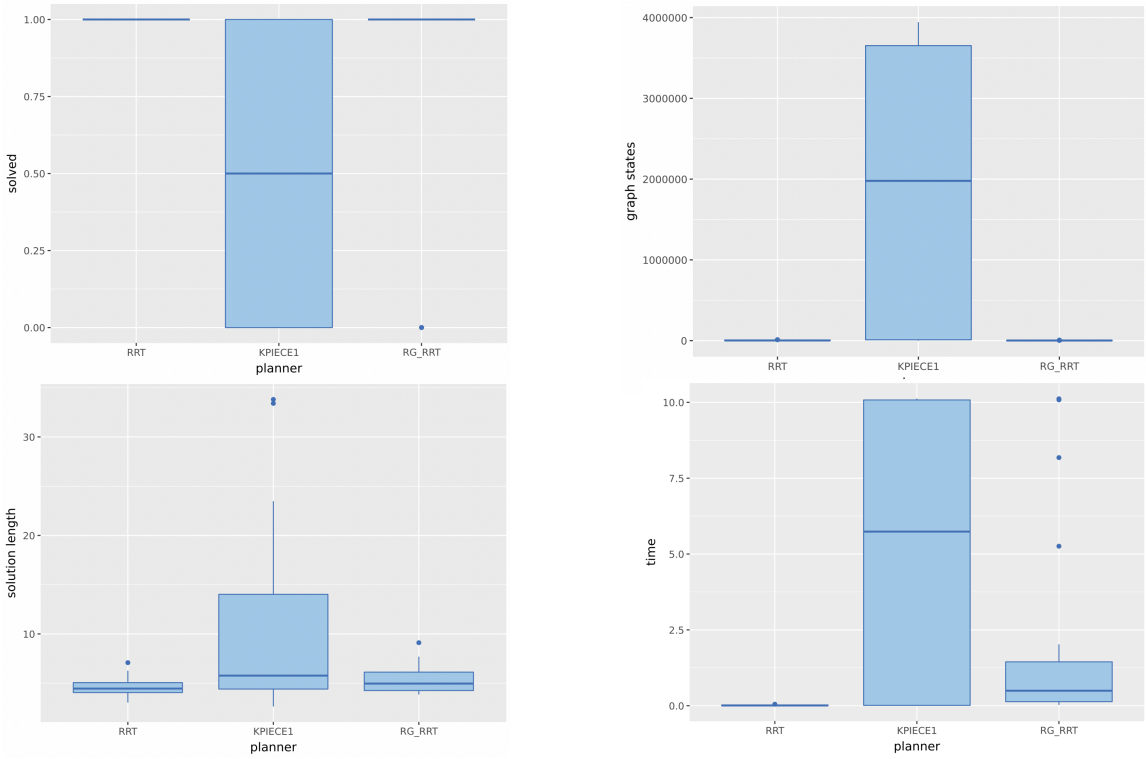


Figure 2: Direct comparison of RRT, KPIECE1, and RG-RRT in the Pendulum System with Low Torque ([-3,3]) via Planner Arena across the following metrics: successfully solved (top left), number states/graph nodes (top right), path length (bottom left), and computation time (bottom right).

In terms of success rate, RRT and RG-RRT outperform KPIECE1, as RRT found a solution for 20/20 runs, and RG-RRT found a solution for 19/20 runs, whereas KPIECE1 only had a median success rate of 50%. When looking at computation time, RRT was the fastest (near-zero), while the median of RG-RRT was relatively low (~0.3 seconds), but had variability caused by the addition of reachability set calculations. Here, again, KPIECE1 underperformed, varying from 0-10 seconds, with a median of ~5.75

seconds. For the number of nodes, RRT and RG-RRT both maintained compact trees (medians of ~1700 and ~900, respectively), whereas KPIECE1 generated extremely large search trees (median of ~2,000,000 graph states). Lastly, looking at solution length, RRT and RG-RRT again performed similarly and better than KPIECE1, as they both had near-zero medians and low data variance (with 75th quartiles between ~0-2), while KPIECE1's median was slightly higher (~1.5), and 75th quartile at ~14. Overall, KPIECE1 was the weakest planner in the torque-controlled pendulum system, which can be explained by its grid projection mechanism. In underactuated systems, KPIECE1 can overestimate coverage of cells, which would assume some states were explored when the system does not have the necessary angular velocity to reach them. Additionally, KPIECE1 scores its grids, which is computationally time-consuming in kinodynamic systems. Moreover, KPIECE1 cannot identify valid energy sequences under tight torque limits, so it unnecessarily repeatedly samples the controls. In contrast, RRT and RG-RRT both performed well in this system. Since this system's state space is only 2-dimensional, Voronoi bias is efficient and the difference between the efficiency of RRT's general algorithm and RG-RRT's reachability set feature is small. RRT succeeds because it rapidly explores the small space, and RG-RRT succeeds because it discards unattainable states and maintains an ultra-compact tree.

Car-Like System:

The car-like system is a kinodynamically-constained system with 2-dimensional control space of angular velocity (heading velocity) and linear acceleration (acceleration of the driving wheels to translate the car in the workspace). In these experiments, the car is represented by a point in the 2-dimensional workspace. The workspace consists of a series of rectangular obstacles along with a start and goal area. This system was tested with OMPL's KPIECE1 and RRT implementations along with our implementation of RG-RRT. Below are the generated paths for each planner:

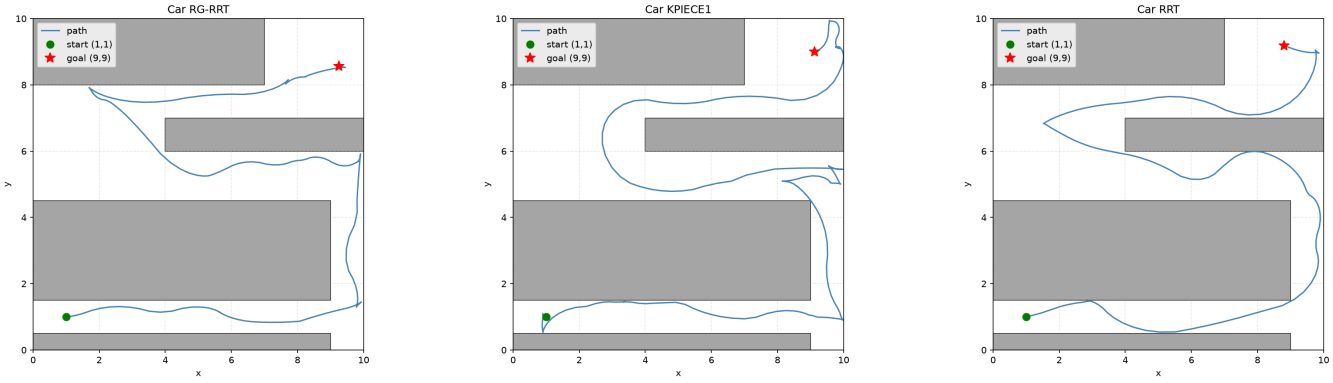


Figure 3: Visualizations of car paths found by KPIECE1, RRT, and RG-RRT Planners.

To compare the performance of KPIECE1, RRT, and RG-RRT for the car-like system, we visualized 20 runs of benchmarking data using Planner Arena. This can be found in figure 4 below. All three planners were able to find solutions for all 20/20 runs. As for graph states, KPIECE1 solutions had significantly more than both RRT and RG-RRT. Although the minimum number of graph states was similar to that of RRT (~800 graph states), it varied widely, resulting in a median of ~34,000 graph states, about 10x the graph states of RRT and RG-RRT. Despite RG-RRT's advantage in terms of graph states for the pendulum robot, the car-like robot showed different results. While the RG-RRT planner had a lower minimum number of graph states for the pendulum, RG-RRT and RRT both produced graph state medians

of ~3300 graph states, indicating that RG-RRT's improvement of RRT in this system was less than that of the pendulum robot. As for solution length, RRT, RG-RRT, and KPIECE1 all performed similarly with median solution lengths of ~33, though the distribution of KPIECE1 shows a number of solutions with lengths of 5-10 more than the maximum lengths of RRT and RG-RRT. For execution time, RRT performed significantly better than KPIECE1 and RG-RRT with a median of 0.023s, 10x less than the other planners. KPIECE1 and RG-RRT yielded similar execution times with medians of ~0.260s and ~0.190s, respectively, with RG-RRT having the slight edge. As in the pendulum system, KPIECE1 was the weakest planner using over 10 times the number of graph states and increased computation time. KPIECE1's advantage of planning without a distance function for handling complex models does not benefit this simpler environment, and instead adds additional complexity without improving the solution paths.

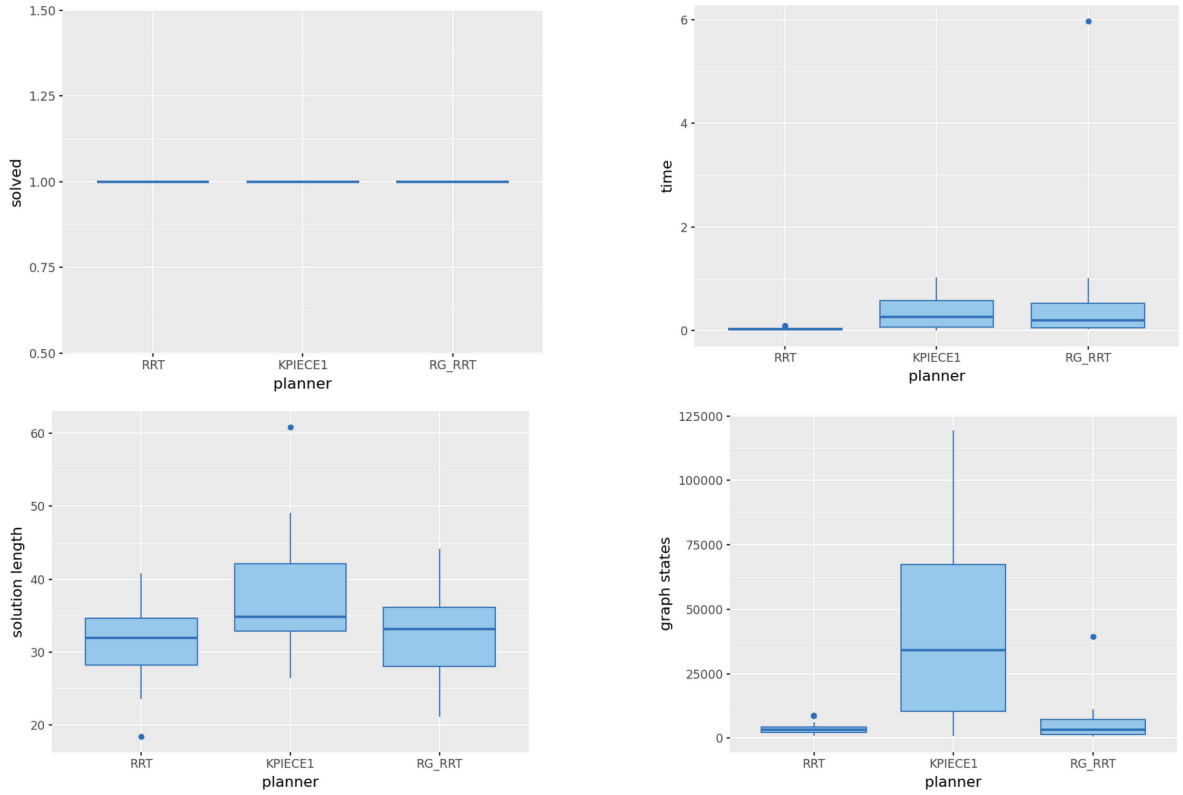


Figure 4: Direct comparison of RRT, KPIECE1, and RG-RRT for the car-like system via Planner Arena across the following metrics: successfully solved (top left), number states/graph nodes (top right), path length (bottom left), computation time (bottom right).

RRT and RG-RRT Comparison:

Though RRT and RG-RRT performed similarly in both systems, they still maintained their differences. In terms of the number of controls, standard RRT picks only one control input per time step, whereas RG-RRT samples a specific constant number of control inputs (>1) in the control space's domain to estimate the reachability set. This feature of RG-RRT caused it to take slightly longer to find solutions in both the pendulum system (median time of ~0.3 seconds RG-RRT vs. near-zero RRT) and the car system (median of ~ 0.023s RRT vs. median of ~0.190 RG-RRT). However, this feature of RG-RRT

produced smoother and more consistent trajectories in both environments (as presented above), as it rejects samples that are outside of the reachability set. Therefore, increasing RG-RRT's number of control inputs would produce smoother trajectories, but also increase the solve time, while decreasing the number of control inputs (until the number is 1) would cause the behavior of RG-RRT to converge to the behavior of standard RRT in terms of path smoothness and solve time.

In terms of the length of the time period, both RG-RRT and RRT have the same value for the time step in our implementation. However, while RRT simply defines the time period as the time duration of forward simulation, RG-RRT defines the time period as the outer bound of the reachability set. Therefore, as can be seen via the trajectory graphs of both systems, the time period defines the length of the curve between each two points and RRT's trajectory is slightly rougher and erratic than RG-RRT. Also, in both environments, both planners yielded similar solution length results (~33 median for car, and ~4-5 median for pendulum). Therefore, theoretically, decreasing the time period would heighten the erratic behavior of RRT and create smoother RG-RRT trajectories. However, decreasing the time period length would also increase the solve time and solution length for both planners, as both would take longer to find nearby states to add to their trees and would take more steps to cover the state space.

Difficulty ratings: Maya:

Exercise 1 was 5/10 in difficulty for me, because although it required translating the material we learned in class into C++ and a significant amount of debugging, it was not very conceptually challenging. It took me 6 hours to complete. Exercise 2 was a 3/10 in difficulty and took me ~1 hour to complete, since the code was already in place. Exercise 3 was a 9/10 in difficulty due to the huge learning curve, and took me 5.5 hours complete. Exercise 4 was a 4/10 in difficulty and took 1 hour to complete. The hardest part of this assignment for me was writing the script for the RG-RRT planner, because it was very time-consuming to learn how to apply the concepts learned in class and presented in the paper to the specific environments in this project.

Difficulty ratings: Tim:

I would say Exercise 1 was a 6/10 for me because I spent a lot of time debugging and getting a better understanding of how OMPL works for kinodynamic planners compared to geometric. In total, it also took me about 6 hours. Exercise 2 was a 5/10; it took less time, but I had to take some time to fully understand how to use projections for KPIECE1. In total, Exercise 2 took about 2 hours. Exercise 3 was 8/10 difficulty because it took me a few rereads of the paper's algorithm descriptions to fully understand how to implement.; I spent about 8 hours. Exercise 4 was 3/10 difficulty because implementing RG-RRT gave me a better understanding of its computational downsides, so comparison came more naturally. I did have to gather additional data relating to the time step and control numbers, so I spent about 4 hours on Exercise 4. Overall, the hardest part of the assignment was the RG-RRT planner.

Contributions:

Maya: Torque-Controlled Pendulum planning with RRT, KPIECE1, and RG-RRT with benchmark comparison. Corresponding section of the paper.
Tim: Car-like system planning with RRT, KPIECE1, and RG-RRT with benchmark comparison. Corresponding section of the paper.
Both: RG-RRT implementation. RRT vs RRG-RRT analysis when varying time step and number of controls.